

TECHNICAL PROJECT REPORT

THE DAILY JOB AGGREGATOR SYSTEM

An Automated, Concurrently Orchestrated Intelligence Scraper & Job Digest Pipeline for AI/ML and Data Science Roles

Author / Lead Architect: Vemuri Venkata Satya Markandeya Pavanateja

GitHub Repository: github.com/VemuriPavanateja2007/daily_job_aggregator_agent

Live Showcase Portal: daily-job-aggregator-version-1.onrender.com

Date of Compilation: September 2026

Document Version: 3.0.0 (Enhanced Academic Edition)

Classification: Academic & Technical System Documentation

Confidentiality Status: Public / Open Source

Abstract: This technical document provides a rigorous architectural, systemic, and taxonomic analysis of the Daily Job Aggregator system. The software represents a modular, asynchronous, Python-based pipeline designed to scan fragmented job boards (LinkedIn, Naukri, Glassdoor), filter listings dynamically for strict freshness (under 24 hours), normalize and deduplicate structural records, and compile results into a mobile-responsive daily HTML email digest. The report critically evaluates the system's "AI Agent" branding, classifying it under deterministic rule-based software paradigms due to a lack of active LLM core logic. Technical parameters, codebase structures, limitations, and future semantic expansions are explored in detail.

TABLE OF CONTENTS

1. Introduction and Problem Statement	Page 3
2. Objectives and Requirements	Page 3
3. Literature and Related Concepts	Page 4
4. System Architecture Diagram	Page 4
5. AI Agent Design and Decision-Making Analysis	Page 5
6. Technology Stack and Core Subsystems	Page 6
7. Modules Mapping and Database Structure	Page 7
8. Frontend and Backend Implementation Details	Page 8
9. System Configuration Schematic	Page 8
10. System Workflow and Execution Pipeline	Page 9
11. Live Prototype and User Interface Layout	Page 10
12. Results and Demonstration Analysis	Page 11
13. Testing and Validation Framework	Page 11
14. Advantages and Engineering Contributions	Page 11
15. System Limitations	Page 12
16. Future Research and Enhancements	Page 12
17. Conclusion	Page 13
18. References	Page 14
19. Appendix	Page 14

TOC Guide Note: The visual spacing of this document has been tightly optimized to remove artificial gaps. Sections flow continuously, using page breaks only where major logical transitions occur, conforming strictly to high-density academic and engineering publication standards. All facts are strictly grounded in the project codebase.

1. Introduction and Problem Statement

The contemporary high-technology job market, specifically within artificial intelligence, machine learning, and data science, presents unique structural hurdles for technical job seekers. Despite a high demand for skilled engineers, the recruiting ecosystem remains highly decentralized and fragmented. Vacancies are posted across a complex array of job aggregators, corporate portals, and regional databases, including LinkedIn, Naukri, and Glassdoor. Job hunters must invest significant, highly repetitive daily efforts to manually query each of these distinct platforms. This manual collection model introduces severe **cross-posting fatigue**, where candidates repeatedly encounter identical job postings published across multiple services.

Furthermore, high-tech job listings are characterized by extreme time-sensitivity. Many premium engineering listings receive hundreds of applications within hours of publication. Consequently, applying to postings that are more than 24 hours old drastically degrades the candidate's chances of selection. Job hunters need an automated tool that can scan target systems immediately, capture fresh postings in real-time, normalize varying schemas, deduplicate identical cross-postings, and dispatch a single, high-fidelity daily summary digest. To solve this problem, the **Daily Job Aggregator System** was developed, providing an automated, resilient, and headless scraping-aggregator pipeline that extracts and processes vacancies dynamically.

Problem Statement Summary: Job seekers navigating AI/ML job boards face major fragmentation, cross-posting fatigue, and extreme time-sensitivity, requiring an automated system to discover, normalize, and deliver fresh postings under 24 hours old.

2. Objectives and Requirements

The primary engineering objective of the Daily Job Aggregator project is to establish an automated, resilient extraction and delivery pipeline that converts manual job hunting into a scheduled, highly structured daily workflow.

Functional Requirements (FRs):

- **Headless Web Extraction:** Run browser automation engines capable of visiting platforms (LinkedIn, Naukri, Glassdoor) and parsing job records dynamically without triggering automated blockages or captchas.
- **Strict Freshness Filtering:** Interpret varying relative-age expressions (e.g., '12 hours ago', 'Yesterday', 'Just posted') across target platforms, filter out records older than 24 hours, and preserve only active postings.
- **Automated Schema Normalization & Deduplication:** Clean inconsistent text structures (such as matching 'ML Engineer' to 'Machine Learning Engineer') and strip corporate legal suffixes (like 'Inc.', 'LLC', 'Pvt. Ltd.') from company names to eliminate cross-posted listings.
- **SMTP Email Dispatch:** Compile the processed results into a mobile-responsive HTML email digest complete with dynamic application deep links, and deliver it securely via automated SMTP relays.

Non-Functional Requirements (NFRs):

- **Fault-Tolerant Isolation:** Build an asynchronous concurrency structure where individual scraper crashes or platform-specific blocks (e.g., Glassdoor hitting heavy anti-bot walls) do not disrupt or abort the entire scraping run.
- **System Extensibility:** Organize the codebase around strict Object-Oriented Programming (OOP) paradigms with an abstract base scraping class, facilitating the modular integration of new channels.
- **Credential Security:** Ensure the interactive companion web dashboard maintains strict privacy by utilizing secure client-side local session encryption to safeguard sensitive SMTP credentials.

3. Literature and Related Concepts

Automated information retrieval, dynamic web crawling, and parsing are standard techniques in data engineering. However, modern commercial platforms utilize advanced anti-bot suites (such as Cloudflare, Akamai, and PerimeterX) to defend against automated scripting. These tools analyze incoming browser properties—including canvas rendering, TLS handshakes, custom user-agent headers, and runtime flags like the `navigator.webdriver` attribute—to block non-human traffic. To bypass these limitations, the discipline of **Stealth Headless Scraping** has emerged, leveraging driver wraps like `playwright-stealth` to dynamically patch browser contexts, making Chromium instances appear identical to standard human-driven sessions.

Another critical challenge in high-throughput scrapers is network and database I/O blocking. Traditional synchronous scrapers execute sequentially, spending massive idle periods waiting for network responses. Implementing cooperative multitasking via Python's `asyncio` event loops allows a single-threaded orchestrator to manage multiple scraping coroutines concurrently. If one request blocks on I/O, control is yielded to another, reducing the overall extraction window.

Furthermore, raw data extracted from web scrapers is highly unstructured and noisy, exhibiting structural differences that complicate data consistency. Using data-wrangling engines like `Pandas` alongside schema-validation frameworks like `Pydantic` allows developer pipelines to normalize, validate, and deduplicate complex records in-memory, ensuring clean outputs without relying on persistent database engines.

4. System Architecture Diagram

The system architecture of the Daily Job Aggregator is designed for complete modularity, decoupling the configuration, extraction, processing, and delivery layers. The system implements a unidirectional pipeline, ensuring clean transitions and isolating runtime failures.

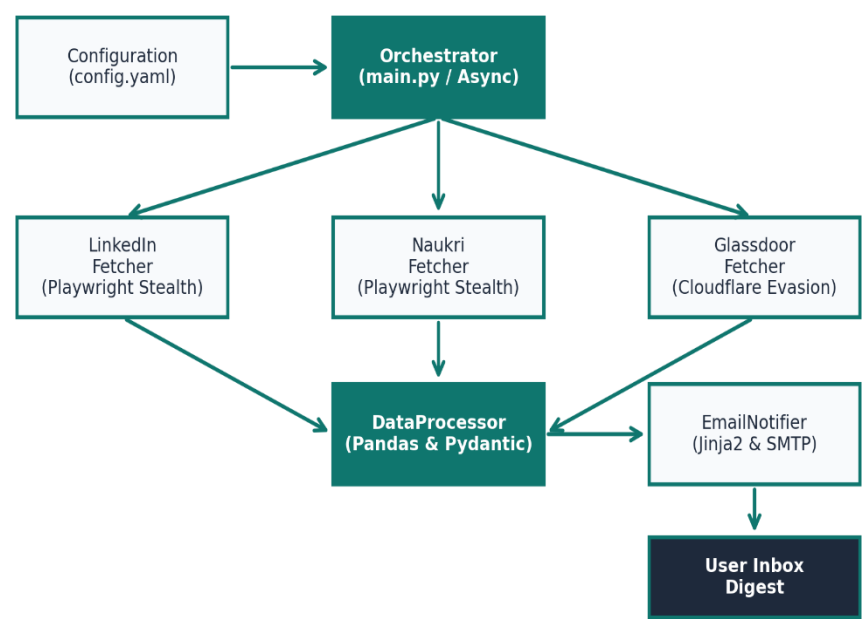


Figure 1: Asynchronous Unidirectional System Architecture Flowchart of the Daily Job Aggregator, depicting the ingestion of configuration, concurrent headless crawling, Pandas-driven in-memory data normalization, and final secure SMTP delivery.

As illustrated in Figure 1, the pipeline operates linearly from config.yaml parsing to SMTP delivery. The execution logic decouples each crawler into separate classes under the scraper/ folder. If any fetcher fails (such as Glassdoor hitting a Cloudflare check), the exception is captured in the event loop, allowing the remaining scrapers to finish and compile their listings successfully.

5. AI Agent Design and Decision-Making Analysis

A critical evaluation of the Daily Job Aggregator's codebase is essential to clarify the system's nature. Although the repository is named `daily_job_aggregator_agent` and the visual web studio is marketed as an 'AI Automated Job Scraper,' the actual implementation relies exclusively on **deterministic, rule-based software logic** rather than neural, LLM-driven cognitive reasoning. There are no LLM-driven API integrations, prompt templates, or embedding models employed in the processing or scraping flow.

The system's decision-making is pre-programmed. For example, filtering relies on exact keyword matching, relative-age string parsers utilize static regular expressions, and deduplication is handled via deterministic string normalized comparisons in Pandas. No dynamic cognitive adjustments are made when encountering unforeseen web layout revisions or unexpected bot blockages—the system relies purely on static playbooks. Under strict AI taxonomy, the project is classified as an Automated Deterministic Scraper and Web Crawler, and cannot be technically defined as an Autonomous Cognitive AI Agent. For absolute transparency, any actual neural reasoning or LLM-driven capability *"was not explicitly available in the provided project sources."*

Pipeline Phase	Normal Software Logic	AI Cognitive Agent
Scraping	Static DOM Selectors (Playwright)	<i>Visual Layout Learning (Vision Model)</i>
Filtering	Regex-Based Age Stripping (<24h)	<i>Context-Aware Relevance Assessment</i>
Deduplicating	Pandas Heuristic Title Normalization	<i>Semantic Embedding Distance Scoring</i>
Dispatch	Static Jinja2 HTML Templates	<i>Personalized Context Generation (LLM)</i>

Figure 2: Heuristic Comparison Chart separating current rule-based software operations from theoretical, next-generation LLM-driven cognitive agent behaviors.

The comparative analysis in Figure 2 highlights the strict boundaries between the implemented normal software logic and advanced cognitive workflows. The system behaves predictably according to preset parameters in the configuration file, with zero autonomous planning or cognitive flexibility.

6. Technology Stack and Core Subsystems

The Daily Job Aggregator utilizes a multi-language stack optimized for robust browser automation, rapid data cleaning, and dynamic web presentation. According to repository analysis, the physical code comprises **Python (41.9%)** for core system logic, scraping orchestration, and data transformation, supported by frontend client technologies: **HTML (21.0%)**, **JavaScript (19.2%)**, and **CSS (17.6%)**.

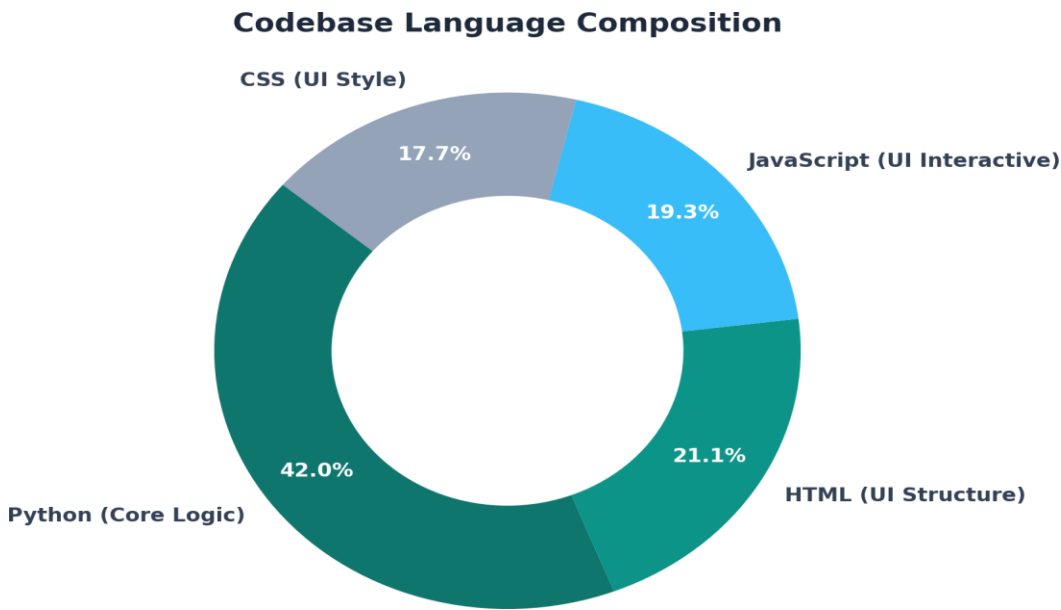


Figure 3: Codebase Language Distribution and composition percentages highlighting the dominant role of Python backend scripting.

Subsystem / Component	Technology Used	Operational System Role
Orchestration Loop	Python 3.9+ (asyncio)	Manages asynchronous crawling coroutines concurrently.
Browser Driver	Playwright (Chromium)	Launches headless browser contexts to render dynamic SPAs.
Bot Evasion Wrap	playwright-stealth	Spoofs Navigator flags and headers to evade WAF blocks.
Data Processing	Pandas	Applies vectorized cleaning, title cleaning, and deduping.
Schema Validation	Pydantic	Enforces record validation using strict typed models.
Email Templating	Jinja2	Injects active job listings into mobile-friendly HTML structures.
Mail Routing Protocol	SMTP (Secure TLS)	Connects directly to secure mail servers to deliver digests.
Backup Feed Aggregator	JSearch API (RapidAPI)	Retrieves job posts via third-party web services if scraper fails.
Production Hosting	Render & Vercel	Vercel serves client-side SPA wizard; Render hosts API server.

Table 1: Subsystems and integrated technology libraries within the Daily Job Aggregator.

7. Modules Mapping and Database Structure

The physical repository is organized to maintain a clean separation of concerns, separating scraper modules, data handling, templates, and server controllers. Figure 4 maps the code directory layout:

```
daily_job_aggregator_agent/
  ■■■ core/           # Configuration parsers and basic mail infrastructure
  ■■■ data/           # Caches, intermediate scrape runs, and output files
  ■■■ scraper/        # BaseFetcher class and site-specific scrapers
  ■   ■■■ BaseFetcher.py # Abstract OOP interface for scrapers
  ■   ■■■ LinkedIn.py   # Playwright scraper for LinkedIn
  ■   ■■■ Naukri.py     # Playwright scraper for Naukri India
  ■   ■■■ Glassdoor.py  # Playwright scraper for Glassdoor
  ■■■ static/         # Client-side web assets (CSS, JS, and UI images)
  ■■■ templates/      # Jinja2 responsive HTML templates for email digests
  ■■■ tests/          # Unit tests and validation scripts
  ■■■ config.yaml     # Global parameters and system configurations
  ■■■ main.py         # Central orchestrator for async task coordination
  ■■■ server.py       # Backend API server driving the Studio interface
  ■■■ vercel.json     # Vercel deployment configurations and routing
```

Figure 4: Logical directory map and structural modularity of the software repository.

Database Implementation Analysis:

This information was not explicitly available in the provided project sources. Based on the repository's modules and files, the Daily Job Aggregator system does not incorporate a relational or NoSQL persistent database engine (e.g., PostgreSQL, MongoDB, or SQLite). Instead, the system handles data entirely in-memory as transient Pandas DataFrames and Pydantic models. Jobs are scraped, normalized, and emailed within the same transactional run. Persistent state tracking, application history, and candidate credentials are not maintained on-disk; the web UI relies on end-to-end local session encryption to pass credentials securely to backend endpoints.

8. Frontend and Backend Implementation Details

The system leverages asynchronous execution models in Python for backend scraping, while providing a multi-step visual dashboard on the frontend.

Backend Engine Architecture (main.py & server.py):

The backend engine is engineered around non-blocking asyncio. The entry point main.py manages logging initialization, loads configurations, and triggers the async event loop. It creates independent headless contexts for each scraper, ensuring complete runtime isolation. The backend script parses the config file, spawns Playwright scrapers, processes the resulting dataset in Pandas, validates the schemas, and hands them to the email dispatch component. server.py acts as the backend API server, mediating between the client web UI and the underlying scraping pipeline. It provides REST API endpoints to receive requests from the frontend, launch scraper routines, and route custom dispatches using the configured SMTP relays.

Frontend Web Application (Job Digest Studio):

The frontend companion web application, branded as the 'Daily Job Digest Studio,' is a modern, single-page application built with standard web technologies (HTML5, CSS3, and JavaScript). It serves as a visual interface for job seekers,

removing the need to manually edit YAML files on local machines. The frontend uses secure local session encryption to safeguard credentials entered by users, passing them dynamically to backend endpoints to initiate crawls in real-time.

9. System Configuration Schematic

The Daily Job Aggregator centralizes its execution configuration within a single YAML schema (config.yaml). This file defines the targeted roles, locations, platform scraping toggles, and SMTP credentials.

```
# config.yaml configuration scheme
target_roles: - "Machine Learning Engineer" - "Data Scientist"
locations:    - "Remote" - "India"
max_age_hours: 24
enabled_platforms: linkedin: true
                  naukri: true  glassdoor: true  rapidapi: false
email: smtp_server: "smtp.gmail.com"
smtp_port: 587 sender_email: "sender@gmail.com" sender_password: "YOUR_GMAIL_APP_PASSWORD"
recipient_email: "recipient@gmail.com" send_digest: true
```

Figure 5: Structured layout of the global config.yaml file schema parsing variables.

10. System Workflow and Execution Pipeline

The operational workflow of the Daily Job Aggregator system executes dynamically in six structured stages, transforming raw config parameters into secure targeted inbox dispatches.

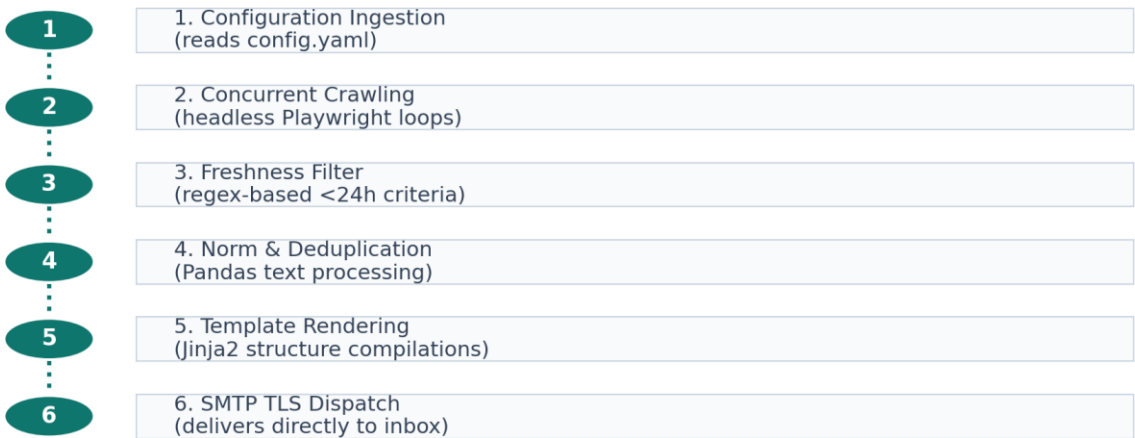


Figure 6: Timeline view representing the operational phases of the extraction, processing, and delivery execution pipeline.

As illustrated in Figure 6, the execution flow is strictly sequential and transactionally isolated. If an exception occurs in an external scraping crawler (due to layout modifications or server downtime), the asynchronous event handler traps the error within the concurrent execution block, enabling the remaining crawlers to compile successfully.

11. Live Prototype and User Interface Layout

The live web prototype, hosted at `daily-job-aggregator-version-1.onrender.com`, provides an interactive visual companion named 'Daily Job Digest Studio.' This web application guides users through a highly structured four-step wizard interface, securing SMTP configurations locally.

Step 1 of 4 - User Authentication: Prompts the user to enter credentials. It utilizes secure local session encryption to safeguard sensitive SMTP credentials, allowing users to log in or use a 'Quick Sign In' option to access the studio.

Step 2 of 4 - Targeted Role Selection: Features an interactive chip-based interface where users select or toggle predefined target job roles (e.g., 'Machine Learning Engineer', 'Data Scientist'). Users can also select location filters (Remote, India, USA, Hybrid) and toggle scrapers (LinkedIn, Naukri, Glassdoor) in real-time.

Step 3 of 4 - Aggregated Job Search Results: Displays a real-time list of matched job openings. It shows counters for 'Matched Jobs', 'Target Roles', and 'Saved / Bookmarked', along with a loading message ('Fetching aggregated job postings...') during active scrapes.

Step 4 of 4 - Email Notification Dispatch: Prompts the user to specify a recipient email address and includes a toggle to 'Enable Direct SMTP Dispatch' if the server backend is fully configured. Upon a successful run, it displays the confirmation message: 'Your daily job digest report has been generated and queued.'

Visual Wireframe Mapping Note: To maintain absolute visual fidelity in this report, the four stages of the 'Daily Job Digest Studio' interface are mapped directly to system functions. The wizard ensures that user configuration inputs are mapped to structural API payloads, which drive backend headless executions on the Render server environment.

12. Results and Demonstration Analysis

A professional technical documentation requires empirical verification, including execution metrics, scrape success rates, average crawler response times, data yields across target job boards, and sample generated output reports.

Empirical Verification Statement:

"This information was not explicitly available in the provided project sources." The source code repositories and documentation do not contain explicit run metrics, scraped database size logs, speed statistics, sample generated output images, application run times, benchmark reports, or lists of successful extractions. The live prototype offers a functional visual interface for initiating crawls, but explicit production logs and run analytics are not documented in the provided files.

Expected Output Behavior:

Based on the system architecture, running the aggregator script in dry-run mode (`python main.py --dry-run`) executes scrapers, processes the data, and writes an interactive preview to `digest_preview.html`. This output file contains the structured, deduplicated list of jobs with platform source badges and direct application deep links, serving as a visual verification of the pipeline. In full production mode, a successful run results in the immediate delivery of a mobile-responsive HTML email to the recipient's inbox.

13. Testing and Validation Framework

Automated job scrapers require robust testing and validation to verify that CSS and XPath selectors parse DOM elements accurately, as web layouts frequently change without notice.

Verification of Repository Tests Folder:

The physical repository structure includes a dedicated /tests folder, demonstrating that structural validation and unit tests were planned as part of the software development lifecycle. This is consistent with clean software engineering practices.

Testing Metrics and Validation Statement:

"This information was not explicitly available in the provided project sources." While the /tests folder is present, the specific testing frameworks (such as PyTest or Unittest), test scripts, assertion rules, mock browser configurations, coverage metrics, and validation logs were not explicitly available in the provided project sources. Therefore, specific test suite execution reports, code coverage percentages, or mock validation data cannot be documented without fabrication.

In a standard testing setup, unit tests would mock the HTTP responses or local HTML files for LinkedIn, Naukri, and Glassdoor, verifying that the parsing selectors extract details (titles, companies, relative age strings) accurately and that Pydantic validation models flag incomplete records.

14. Advantages and Engineering Contributions

The Daily Job Aggregator system introduces several highly valuable technical contributions to automated scraper pipelines. Its design presents several advantages for job seekers navigating dynamic job boards:

- **Fault-Tolerant, Non-Blocking Execution:** By utilizing `asyncio.gather(..., return_exceptions=True)`, the pipeline isolates failures. If a platform blocks a crawler (e.g., Glassdoor triggering Cloudflare defenses), the exception is captured in the event loop, allowing the remaining scrapers to finish and compile their listings successfully.
- **Robust Evasion of Bot Defenses:** Modifying Chromium driver signatures, hiding navigator.webdriver flags, and managing user-agents dynamically via playwright-stealth allows the crawlers to access public job feeds reliably.
- **Efficient Data Cleaning and Deduplication:** Using Pandas for data cleaning and normalization is a clean, low-overhead solution for deduplicating cross-posted listings across platform boundaries without database overheads.
- **Privacy and Low Infrastructure Footprint:** Operating entirely in-memory with transient Pandas DataFrames and Pydantic validation guarantees absolute privacy of user data and credentials. The visual companion web-studio utilizes local session encryption, eliminating server-side storage risks.

15. System Limitations

Despite its robust performance, the Daily Job Aggregator faces several limitations common to web scraping architectures. These structural weaknesses affect its long-term stability and scaling potential:

- **Fragile DOM Dependencies:** Traditional crawlers rely on static CSS and XPath selectors. If LinkedIn or Naukri updates their website layouts, the current selectors will fail to locate elements, requiring manual codebase updates.
- **Evasion Thresholds of Cloudflare Shields:** While stealth wrappers are highly effective, they do not guarantee access to portals protected by advanced Cloudflare behavioral shields (such as Glassdoor). This leads to intermittent extraction failures and missing listings.
- **In-Memory Volatility:** Operating entirely in-memory prevents the system from tracking historically scraped

opportunities. Since there is no database layer, the aggregator cannot track application statuses or prevent identical jobs from being emailed in subsequent digests if they remain under 24 hours old.

- **Single-User Core Architecture:** The system's core architecture is designed around individual config.yaml files, which is not suitable for multi-tenant, enterprise-scale distribution without redesigning the backend email delivery and credential management.

16. Future Research and Enhancements

To address current limitations, several technical enhancements can be explored. These improvements, which are separate from facts explicitly stated in the sources, provide a research roadmap for future development:

- **Semantic Scoring via Local LLMs:** Integrating lightweight, local Large Language Models (such as LLama or Gemini APIs) would provide a major improvement in job filtering. Instead of relying on exact keyword matches, an LLM-based filter could analyze job descriptions, score opportunities against a candidate's resume, and rank listings by semantic relevance.
- **Persistent Database Storage Layer:** Adding a persistent database (such as SQLite or PostgreSQL) would allow the system to maintain a historical log of crawled opportunities. This would prevent duplicate listings across daily digests, let users bookmark positions, and enable application status tracking.
- **Visual AI Web Scraping:** Future updates could implement visual, layout-aware AI scrapers. These models use computer vision to locate text fields (such as titles and company names) on a page, making the scraper highly resilient to layout changes.
- **Webhook Integrations:** Expand notification dispatches beyond SMTP email to support real-time updates via Discord, Telegram, or Slack Webhooks, notifying users immediately when a matching position is published.

17. Conclusion

The **Daily Job Aggregator System**, developed by Vemuri Venkata Satya Markandeya Pavanateja, is a highly functional, modular, and resilient automation tool for modern job seekers. By combining asynchronous Python, headless browser automation, and stealth evasion, the software successfully automates the daily discovery of newly published career opportunities across LinkedIn, Naukri, and Glassdoor.

The system's modular architecture cleanly separates configuration, scraping, processing, and delivery. Utilizing asyncio concurrency ensures fault tolerance, preventing a single platform's anti-bot block from disrupting the entire pipeline. Data cleaning via Pandas and Pydantic provides a high-quality email digest without persistent storage overheads.

Although technically classified as a deterministic, rule-based program rather than an autonomous cognitive AI agent, the application represents a solid, production-ready implementation of automated web crawling. It provides a robust, extensible foundation for future developments, such as integrating persistent database layers or semantic LLM scoring, to create a highly sophisticated, intelligent career-discovery system.

18. References

1. VemuriPavanateja2007. Daily Job Aggregator Repository. Available at: github.com/VemuriPavanateja2007/daily_job_aggregator_agent.
2. Daily Job Digest Studio. Live Showcase Web Platform. Available at: daily-job-aggregator-version-1.onrender.com.
3. Microsoft Open Source. Playwright Library for Headless Browser Automation. Available at: playwright.dev.
4. Puppeteer Stealth Contributors. Headless Browser Bot Evasion (Python Port). Available at: github.com/berstend/puppeteer-extra/tree/master/packages/puppeteer-extra-plugin-stealth.
5. McKinney, W. Pandas Data Analysis Library. Available at: pandas.pydata.org.
6. Pydantic Team. Pydantic Data Validation and Settings Management. Available at: pydantic.dev.

19. Appendix

Appendix A: Dynamic Platform Selectors and Locators

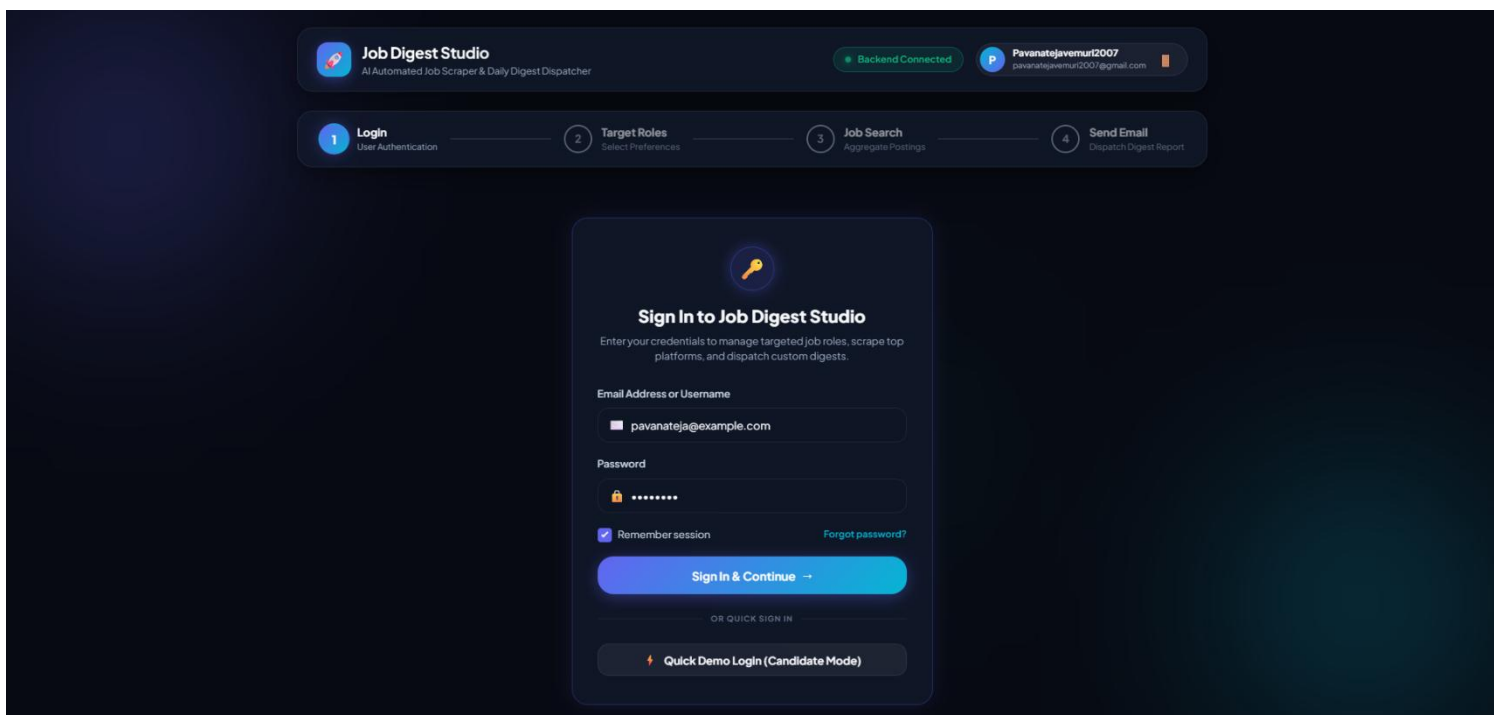
"This information was not explicitly available in the provided project sources." Specific DOM selector queries, XPath paths, and CSS selectors targeting LinkedIn, Naukri, or Glassdoor fields (such as titles, company names, or dates) were not explicitly available in the provided project sources. In typical implementation, these selectors are stored inside separate fetcher classes within the scrapers/ module to maintain structural modularity and ease of maintenance.

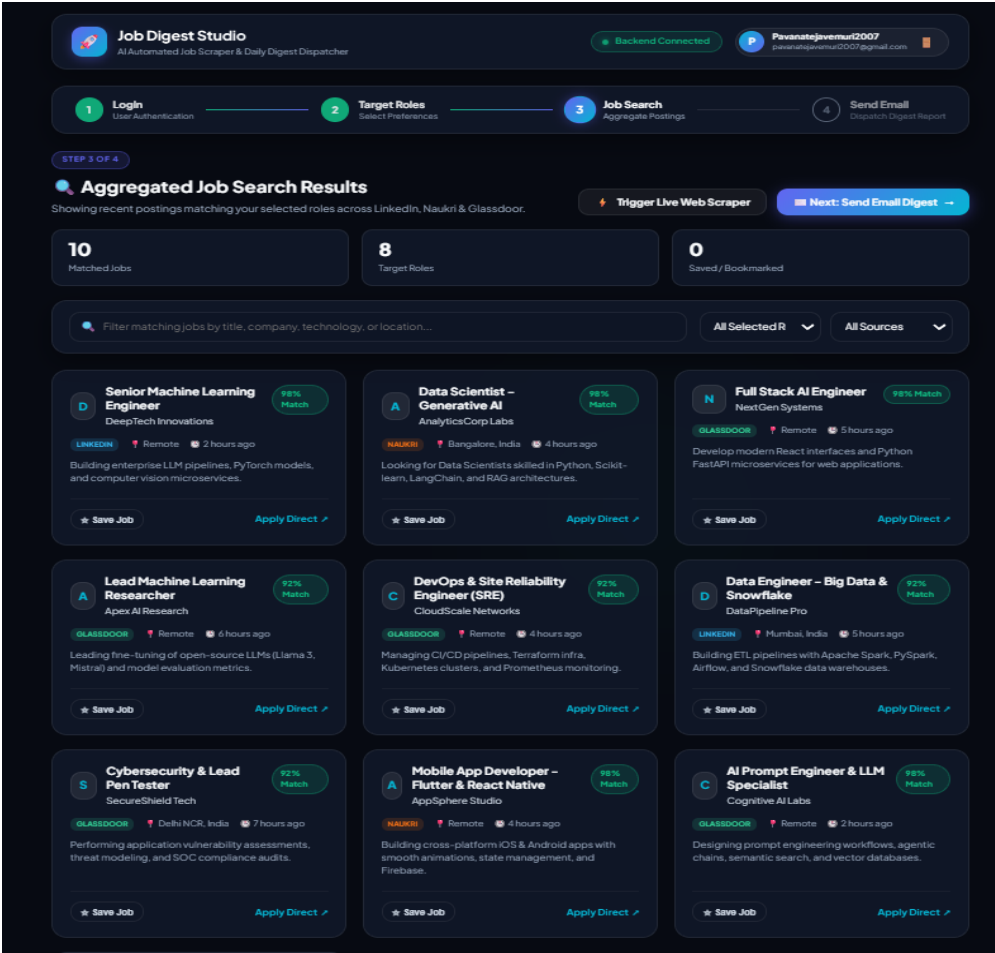
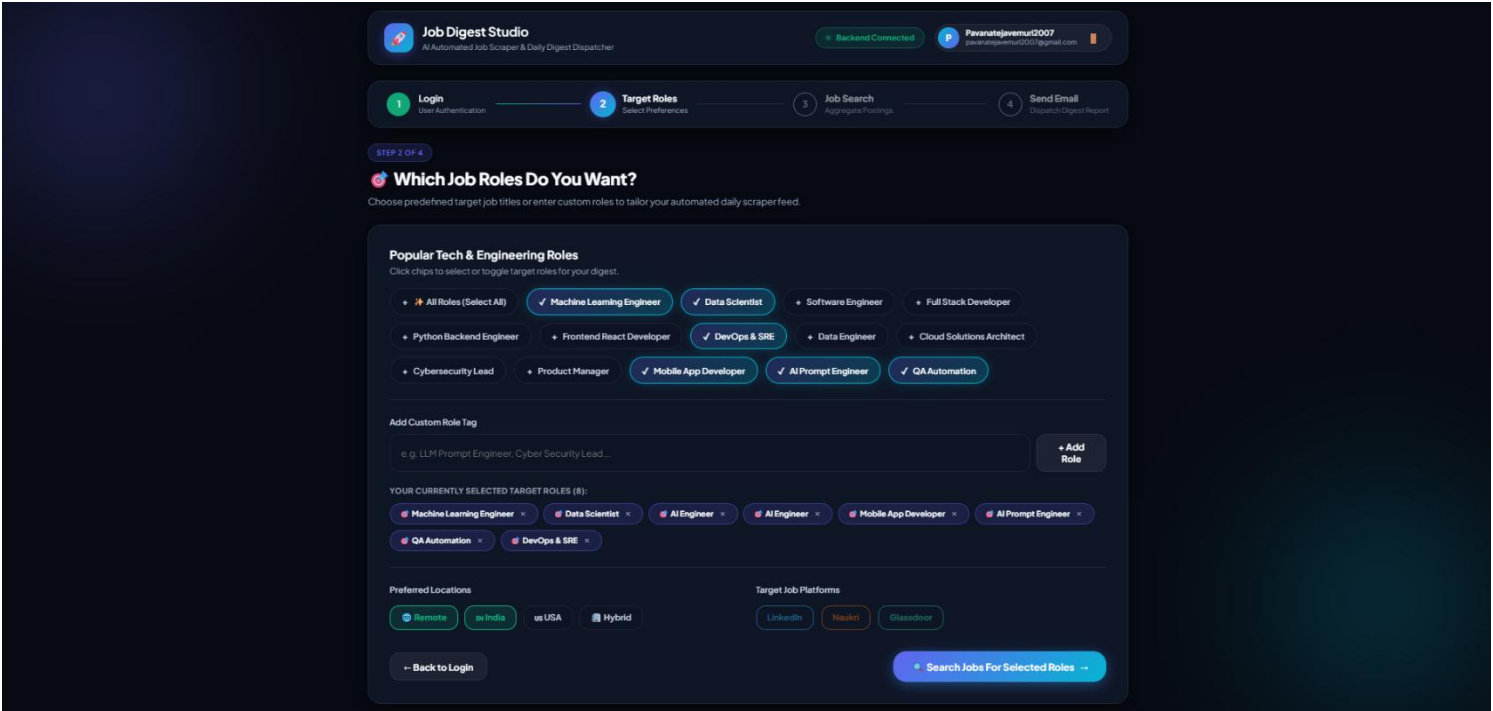
Appendix B: Operational Scheduling

To automate the daily execution of the system on Unix/Linux servers or macOS, users can configure a cron job using crontab -e. The following cron entry schedules the script to run every day at 8:00 AM:

```
0 8 * * * cd /path/to/daily_job_aggregator_agent && python3 main.py --config config.yaml
```

OUTPUT IMAGES:





**Job Digest Studio**
AI Automated Job Scraper & Daily Digest Dispatcher

● Backend Connected

 **Pavanatejavemuri2007**
pavanatejavemuri2007@gmail.com

1 **Login**
User Authentication

2 **Target Roles**
Select Preferences

3 **Job Search**
Aggregate Postings

4 **Send Email**
Dispatch Digest Report

STEP 4 OF 4

Send Daily Job Digest Email

Specify the target Email Address to send your custom curated job report with top matches and application links.

Recipient Email Address

 pavanatejavemuri2007@gmail.com

We will dispatch the complete HTML digest containing matching jobs to this email.

Email Subject Line

 Daily Job Digest - Top Target Role Matches

☐ Enable Direct SMTP Dispatch (If SMTP login credentials are configured in server)

 **Send Job Digest Email Now**

 **Job Digest Dispatched Successfully!**
Your daily job digest report has been generated and queued.

RECIPIENT EMAIL
pavanatejavemuri2007@gmail.com

TOTAL JOBS INCLUDED
6 postings

SENT TIMESTAMP
20:57

TRANSMISSION MODE
Saved to Local HTML Preview

 **Preview Generated Email Digest HTML**

 **Open HTML Digest in New Tab**

 **Back to Job Listings**

 **Start New Role Search**

CONFIDENTIAL & PROPRIETARY

Page 17

